

Code of Conduct

All assignments are graded, meaning we expect you to adhere to the academic integrity standards of NYU Abu Dhabi. To avoid any confusion regarding this, we will briefly state what is and isn't allowed when working on an assignment.

Any documents and program code that you submit must be fully written by yourself and properly commented. You can, of course, discuss your work with fellow students, as long as these discussions are restricted to general solution techniques. Put differently, these discussions should not be about the concrete code you are writing, nor about specific results you wish to submit. When discussing an assignment with others, this should never lead to you possessing the complete or partial solution of others, regardless of whether the solution is in paper or digital form, and independent of who made the solution, meaning you are also not allowed to possess solutions by someone from a different year or course, by someone from another university, or code from the Internet, etc. This also implies that there is never a valid reason to share your code with fellow students and that there is no valid reason to publish your code online in any form.

In the header comments, please include a short paragraph that reflects on your use of AI. If this is the case, explicitly declare the usage of the AI tool, including any prompts that you crafted and responses that you got from the tool, and for which part of the assignment.

Every student is responsible for the work they submit. **If there is any doubt during the grading about whether a student created the assignment themselves** (e.g. if the solution matches that of others), we reserve the option to **let the student explain** why this is the case. We also reserve the right to **randomly select students to explain the solution implemented**. **Reduction points may apply if we detect that the student cannot explain parts of the code submitted**. Please keep in mind that **MOSS will be used** during the grading process. In case doubts remain, or we decide to directly escalate the issue, the suspected violations will be reported to the academic administration according to the internal policies of NYU Abu Dhabi (see <https://students.nyuad.nyu.edu/campus-life/community-standards/policies/academic-integrity>). Please check the **Oral Assessment for Academic Integrity Verification** text shared online on Brightspace under **Course Policies**.

Floating-Point Representation Using Strings

Understanding how floating-point numbers are represented in memory and how to manipulate that representation is crucial to having a clear view of how computer systems work and to writing efficient software. In this exercise, our goal is to understand the floating-point representation and how it is interpreted. We will convert a simple floating number to its binary representation in the form of a string, then we will construct our own representation to consolidate our understanding of the different components that make up a float. In this exercise (tasks 1 and 2), we will mainly use string manipulation functions to solve all the tasks. In the next exercise (tasks 3 to 7), we will mainly use bit manipulation instead of using string manipulation. Throughout the whole assignment, the special values `+0`, `-0`, `+inf`, `-inf`, as well as normalized numbers, should be supported, but NaN and denormalized numbers do not need to be supported.

Download the starter code from Brightspace and complete the following functions declared in the header file called `functions.h`. The starter code also contains `main.c`, which can be used to test your code.

Task 1: `char* float_to_binary(float value)`

Write a function `char* float_to_binary(float value)` that converts a floating-point number into its **IEEE 754 binary representation** as a C-string. This string should represent the full 32-bit binary form of the float.

You must implement the **IEEE floating-point conversion logic yourself**. Do not use any built-in library functions or compiler-provided conversions for this task. You are also not allowed to extract or reinterpret the raw bits from the memory of a float variable. Instead, use **decimal-to-float conversion techniques** to determine the **sign bit, exponent, and mantissa**. Since the intermediate steps may involve large numbers, you may need to use a longer **C-string (character array)** to store results before finalizing the 32-bit representation.

Example:

Given `-168.25` float as input the function returns the following string:

`11000011001010000100000000000000`

(The sign bit is highlighted in yellow, the exponent is highlighted in cyan, and the significand also called fraction, field is highlighted in gray.)

Task 2: `char* switch_significand_exponent_string(char* value)`

Write a function that takes a string representation of a floating-point number and switches the positions of the exponent and the significand. If the string

`11000011001010000100000000000000` is passed to the function, it must return `101010000100000000000000010000110`

Task 3: `char* invert_bit_string(char* value)`

Write a function that takes a string representation of a floating-point number and inverts it. If a string representing the -168.25 float is passed as an input to the function, then it must return:

000000000000000100001010011000011

Floating-Point Representation Using Bit Manipulation

Manipulating floats through their string representation is not efficient or practical. In real-world applications, one would modify the representation of floats using bit manipulation. Bit manipulation is useful beyond the scope of modifying floating-points. It is a powerful tool used in many applications, such as data encryption, compression, image processing, error-correcting codes and set manipulation. We want to perform the previous steps using bit manipulation instead of string manipulation. Given a float as input, write the following functions:

Task 4: `unsigned int get_sign(float value)`

Write a function that extracts the sign bit from the input float using bit manipulation. The function takes a float as input and returns an unsigned int containing the sign bit. The sign bit must be the leftmost bit of the returned value (as shown in the example below).

Example:

Given float -168.25 as input to the function, the function extracts and returns an unsigned int containing the sign bit as its left most bit. For the current case, the result should be:

10000000000000000000000000000000

Task 5: `unsigned int get_exponent(float value)`

Write a function that extracts the exponent from the input float using bit manipulation. The function takes a float as input and returns an unsigned int containing the exponent.

Example:

Given the -168.25 float as input to the function to extract the exponent, the function returns an unsigned integer that has the following binary representation:

00000000000000000000000010000110

Task 6: `unsigned int get_significand(float value)`

Write a function that takes a float as input and extracts the significand from the input float using bit manipulation. The function returns an unsigned int containing the significand.

Example:

Given the -168.25 floats as input, the function returns an unsigned integer that has the following binary representation:

00000000010100001000000000000000

(The significand of -168.25 interpreted as an unsigned integer value should be $2^{21} + 2^{19} + 2^{14} = 2,637,824$)

Task 7: unsigned int switch_significand_exponent(float value)

Write a function that switches the significand and exponent positions in float using bit manipulation. The function takes as input a float and returns an unsigned integer containing the new representation. You may use the previously developed functions (*get_sign()*, *get_exponent()*, *get_significant()*) in this function.

Example:

Given the -168.25 float as input, the function returns an unsigned int which has the following bit representation:

10101000010000000000000010000110

Task 8: unsigned int flip_sign(float value)

Write a function that flips the sign of a float numbers using bit manipulation. The function takes as input a float, reverses the sign and returns an unsigned integer containing the new representation. You may use any of the previously developed functions in this function.

Example:

Given the -168.25 float as input, the function returns an unsigned int which has the following bit representation:

01000011001010000100000000000000

Task 9: unsigned short reduce_FP32_to_FP16(unsigned int value)

Finally, write a function that expands an IEEE 754 FP32 bit number to the equivalent FP16 notation, using bit manipulation. The function takes as input an unsigned int, performs the necessary calculations and bit extensions to the significand and the exponent, and returns an unsigned 16-bit integer containing the new representation. Please note that from FP32 to FP16, the exponent bias changes. Also note that the mantissa may lose precision.

Example:

Given the unsigned int representation of the same -168.25 float previously indicated as input, the function returns a 16-bit unsigned int which has the following bit representation:

1101100101000010

Grading

The assignment is worth 20 points. You can find the points per function in the table below.

Grading	
Task 1: float_to_binary	3
Task 2: switch_significand_exponent_string	2
Task 3: invert_bit_string	2
Task 4: get_sign	1
Task 5: get_exponent	2
Task 6: get_significand	2
Task 7: switch_significand_exponent	2
Task 8: flip_sign	2
Task 9: reduce_FP32_to_FP16	2
Code properly commented (comments should be made on separate lines)	2

Notes

You need to complete the functions declared in the `functions.h`. Please test your solution on a second compiler as well. The goal is to make sure that your solution compiles and runs correctly on different compilers.

Comments

Comments play a crucial role in programming assignments. They help explain the purpose and logic of your code, making it easier for others (including your instructors and peers) to understand your work. Well-written comments demonstrate your understanding of the code and improve its readability and maintainability. Comments will account for 10% of the score in your programming assignments.

Be sure to include meaningful and clear comments that describe the functionality and logic of your code. All comments must be written on a separate line (not on the same line as code) so that the autograder can properly detect and evaluate them.

Posting the Assignment and/or Solutions Online

Posting this assignment and/or the solution of the assignment online is not allowed.

Submission

Please submit a well-commented and complete header file (`functions.h`) to Gradescope. Passing the Gradescope tests does not guarantee full marks; you are still responsible for thoroughly testing your code. Final grades will be posted on the Brightspace gradebook. Submissions via e-mail are not accepted.

Expected Output

1. Normalized number (-168.25)

```
Task 1: The IEEE 754 FP32 binary representation of -168.25 is:
11000011001010000100000000000000

Task 2: After switching the significand and exponent bits:
10101000010000000000000010000110

Task 3: After inverting the bits of 11000011001010000100000000000000:
00000000000000100001010011000011

Task 4: The sign bit of -168.25 is:
10000000000000000000000000000000

Task 5: The exponent bits of -168.25 are:
00000000000000000000000010000110

Task 6: The significand bits of -168.25 are:
00000000001010000100000000000000

Task 7: After switching the significand and exponent bits of -168.25:
10101000010000000000000010000110

Task 8: After flipping the sign bit of -168.25:
01000011001010000100000000000000

Task 9: The IEEE 754 FP16 binary representation of -168.25 is:
1101100101000010
```

2. Normalized number (168.25)

```
bash-3.2$ ./a.out +168.25
Task 1: The IEEE 754 FP32 binary representation of 168.250000 is:
01000011001010000100000000000000

Task 2: After switching the significand and exponent bits:
00101000010000000000000010000110

Task 3: After inverting the bits of 01000011001010000100000000000000:
00000000000000100001010011000010

Task 4: The sign bit of 168.25 is:
00000000000000000000000000000000

Task 5: The exponent bits of 168.25 are:
00000000000000000000000010000110

Task 6: The significand bits of 168.25 are:
00000000001010000100000000000000

Task 7: After switching the significand and exponent bits of 168.25:
00101000010000000000000010000110

Task 8: After flipping the sign bit of 168.25:
11000011001010000100000000000000

Task 9: The IEEE 754 FP16 binary representation of 168.25 is:
0101100101000010
```

3. Positive Zero (+0.0)

```
Task 1: The IEEE 754 FP32 binary representation of 0.00 is:
00000000000000000000000000000000

Task 2: After switching the significand and exponent bits:
00000000000000000000000000000000

Task 3: After inverting the bits of 00000000000000000000000000000000:
00000000000000000000000000000000

Task 4: The sign bit of 0.00 is:
00000000000000000000000000000000

Task 5: The exponent bits of 0.00 are:
00000000000000000000000000000000

Task 6: The significand bits of 0.00 are:
00000000000000000000000000000000

Task 7: After switching the significand and exponent bits of 0.00:
00000000000000000000000000000000

Task 8: After flipping the sign bit of 0.00:
10000000000000000000000000000000

Task 9: The IEEE 754 FP16 binary representation of 0.00 is:
0000000000000000
```

4. Negative Zero (-0.0)

```
Task 1: The IEEE 754 FP32 binary representation of -0.00 is:
10000000000000000000000000000000

Task 2: After switching the significand and exponent bits:
10000000000000000000000000000000

Task 3: After inverting the bits of 10000000000000000000000000000000:
00000000000000000000000000000001

Task 4: The sign bit of -0.00 is:
10000000000000000000000000000000

Task 5: The exponent bits of -0.00 are:
00000000000000000000000000000000

Task 6: The significand bits of -0.00 are:
00000000000000000000000000000000

Task 7: After switching the significand and exponent bits of -0.00:
10000000000000000000000000000000

Task 8: After flipping the sign bit of -0.00:
00000000000000000000000000000000

Task 9: The IEEE 754 FP16 binary representation of -0.00 is:
1000000000000000
```

5. Positive Infinity (+inf)

Task 1: The IEEE 754 FP32 binary representation of inf is:
01111111100000000000000000000000

Task 2: After switching the significand and exponent bits:
00000000000000000000000011111111

Task 3: After inverting the bits of 01111111100000000000000000000000:
000000000000000000000000111111110

Task 4: The sign bit of inf is:
00000000000000000000000000000000

Task 5: The exponent bits of inf are:
00000000000000000000000011111111

Task 6: The significand bits of inf are:
00000000000000000000000000000000

Task 7: After switching the significand and exponent bits of inf:
00000000000000000000000011111111

Task 8: After flipping the sign bit of inf:
11111111100000000000000000000000

Task 9: The IEEE 754 FP16 binary representation of inf is:
0111110000000000

6. Negative Infinity (-inf)

Task 1: The IEEE 754 FP32 binary representation of -inf is:
11111111100000000000000000000000

Task 2: After switching the significand and exponent bits:
10000000000000000000000011111111

Task 3: After inverting the bits of 11111111100000000000000000000000:
000000000000000000000000111111111

Task 4: The sign bit of -inf is:
10000000000000000000000000000000

Task 5: The exponent bits of -inf are:
00000000000000000000000011111111

Task 6: The significand bits of -inf are:
00000000000000000000000000000000

Task 7: After switching the significand and exponent bits of -inf:
10000000000000000000000011111111

Task 8: After flipping the sign bit of -inf:
01111111100000000000000000000000

Task 9: The IEEE 754 FP16 binary representation of -inf is:
1111110000000000

7. Large Positive Normal Number (3.4028234664e38)

```
bash-3.2$ ./a.out 3.4028234664e38
Task 1: The IEEE 754 FP32 binary representation of 340282346638528859811704183484516925440.00 is:
01111111011111111111111111111111

Task 2: After switching the significand and exponent bits:
01111111111111111111111111111110

Task 3: After inverting the bits of 01111111011111111111111111111111:
11111111111111111111111101111110

Task 4: The sign bit of 340282346638528859811704183484516925440.00 is:
00000000000000000000000000000000

Task 5: The exponent bits of 340282346638528859811704183484516925440.00 are:
00000000000000000000000001111110

Task 6: The significand bits of 340282346638528859811704183484516925440.00 are:
00000000011111111111111111111111

Task 7: After switching the significand and exponent bits of 340282346638528859811704183484516925440.00:
01111111111111111111111111111110

Task 8: After flipping the sign bit of 340282346638528859811704183484516925440.00:
11111111011111111111111111111111

Task 9: The IEEE 754 FP16 binary representation of 340282346638528859811704183484516925440.00 is:
0111100000000000
```

8. Smallest Positive Normal Number (1.1754943508e-38)

```
bash-3.2$ ./a.out 1.1754943508e-38
Task 1: The IEEE 754 FP32 binary representation of 0.00 is:
00000000100000000000000000000000

Task 2: After switching the significand and exponent bits:
00000000000000000000000000000001

Task 3: After inverting the bits of 00000000100000000000000000000000:
0000000000000000000000000100000000

Task 4: The sign bit of 0.00 is:
00000000000000000000000000000000

Task 5: The exponent bits of 0.00 are:
00000000000000000000000000000001

Task 6: The significand bits of 0.00 are:
00000000000000000000000000000000

Task 7: After switching the significand and exponent bits of 0.00:
00000000000000000000000000000001

Task 8: After flipping the sign bit of 0.00:
10000000100000000000000000000000

Task 9: The IEEE 754 FP16 binary representation of 0.00 is:
0000000000000000
```